

ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ОБРАЗОВАНИЮ

ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ «САМАРСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ имени академика С.П. КОРОЛЁВА»

КАФЕДРА «ТЕХНИЧЕСКАЯ КИБЕРНЕТИКА»

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ

«Объектно-ориентированное программирование»

ЛАБОРАТОРНАЯ РАБОТА №7

Студент _____ Волков Д.Н.

Группа _____ 6301-030301D

Руководитель _____ Борисов Д. С.

Оценка _____

Задание №1

В интерфейсе TabulatedFunction добавил необходимый родительский тип.

```
package functions;

public interface TabulatedFunction extends Function, Cloneable, Iterable<FunctionPoint> {
    int getPointsCount();
    FunctionPoint getPoint(int index);
    void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException;
    double getPointX(int index);
    void setPointX(int index, double x) throws InappropriateFunctionPointException;
    double getPointY(int index);
    void setPointY(int index, double y);
    void deletePoint(int index) throws InappropriateFunctionPointException;
    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;
    TabulatedFunction clone();
}
```

В классе ArrayTabulatedFunction добавил метод возвращающий объект итератора, класс итератора сделал анонимным.

```
@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private int index = 0;

        @Override
        public boolean hasNext() {
            return index < pointsCount;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException(s: "Следующего элемента нет");
            }
            return new FunctionPoint(points[index++]);
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException(message: "Удалить элемент нельзя");
        }
    };
}
```

Метод удаления выбрасывает исключение UnsupportedOperationException.

Метод получения следующего элемента выбрасывает исключение NoSuchElementException, если следующего элемента нет.

В LinkedListTabulatedFunction реализовал аналогичный итератор.

```
@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private FunctionNode node = head.getNext();

        @Override
        public boolean hasNext() {
            return node != head;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException(s: "Следующего элемента нет");
            }
            FunctionPoint point = new FunctionPoint(node.getPoint());
            node = node.getNext();
            return point;
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException(message: "Удалить элемент нельзя");
        }
    };
}
```

В методе main программы проверил работу итераторов классов табулированных функций.

```
public static void main(String[] args) throws IOException, FunctionPointIndexOutOfBoundsException, Inappropri

    double[] values = {1, 5, 3, 10, 2};
    ArrayTabulatedFunction arrayFunc1 = new ArrayTabulatedFunction(leftX: 1, rightX: 15, values);
    LinkedListTabulatedFunction listFunc1 = new LinkedListTabulatedFunction(leftX: 1, rightX: 15, values);

    System.out.println(x: "\nArrayTabulatedFunction\n");
    for (FunctionPoint p : arrayFunc1) {
        System.out.println(p);
    }
    System.out.println(x: "\nLinkedListTabulatedFunction\n");
    for (FunctionPoint p : listFunc1) {
        System.out.println(p);
    }
}
```

```
ArrayTabulatedFunction

(1.0; 1.0)
(4.5; 5.0)
(8.0; 3.0)
(11.5; 10.0)
(15.0; 2.0)

LinkedListTabulatedFunction

(1.0; 1.0)
(4.5; 5.0)
(8.0; 3.0)
(11.5; 10.0)
(15.0; 2.0)

-----
```

Задание №2

В пакете functions описал базовый интерфейс фабрик табулированных функций TabulatedFunctionFactory. В интерфейс добавил три перегруженных метода.

```
package functions;

public interface TabulatedFunctionFactory {
    TabulatedFunction createTabulatedFunction(FunctionPoint[] points);
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount);
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values);
}
```

Описал в ArrayTabulatedFunction и LinkedListTabulatedFunction классы фабрик.

```
public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory{

    @Override
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new ArrayTabulatedFunction(points);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

}
```

```

public static class LinkedListTabulatedFunctionFactory implements TabulatedFunctionFactory{

    @Override
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new LinkedListTabulatedFunction(points);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new LinkedListTabulatedFunction(leftX, rightX, values);
    }
}

```

В классе TabulatedFunctions объявил приватное статическое поле типа TabulatedFunctionFactory и проинициализировал его объектом ArrayTabulatedFunction фабрик.

```

public final class TabulatedFunctions {

    private static TabulatedFunctionFactory factory = new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory();
}

```

Также объявил метод setTabulatedFunctionFactory, позволяющий заменить объект фабрики.

```

public static void setTabulatedFunctionFactory(TabulatedFunctionFactory factory) {
    TabulatedFunctions.factory = factory;
}

```

В классе TabulatedFunctions описал три перегруженных метода TabulatedFunction createTabulatedFunction, возвращающих объекты табулированных функций, созданные с помощью текущей фабрики.

```

public static TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
    return factory.createTabulatedFunction(points);
}

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
    return factory.createTabulatedFunction(leftX, rightX, pointsCount);
}

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
    return factory.createTabulatedFunction(leftX, rightX, values);
}

```

В остальных методах (tabulate, inputTabulatedFunction, readTabulatedFunction) класса заменил явное создание объектов с помощью конструкторов на вызов метода createTabulatedFunction.

```
return createTabulatedFunction(points);
```

В методе main проверил работу фабрик.

```
-----  
class functions.ArrayTabulatedFunction  
class functions.LinkedListTabulatedFunction  
class functions.ArrayTabulatedFunction  
-----
```

Задание №3

В классе TabulatedFunctions добавил ещё три перегруженных версии метода createTabulatedFunction. Эти методы получают ссылку типа Class на описание класса, объект которого требуется создать. Сделал так, чтобы в эти методы можно было передать только ссылки на классы, реализующие интерфейс TabulatedFunction.

```
public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> Class, FunctionPoint[] points) {  
    try {  
        return Class.getConstructor(...parameterTypes: FunctionPoint[].class).newInstance((Object) points);  
    } catch (NoSuchMethodException e) {  
        throw new IllegalArgumentException("Не найден конструктор " + e.getMessage());  
    } catch (IllegalAccessException | IllegalArgumentException | InstantiationException | InvocationTargetException e) {  
        throw new IllegalArgumentException("Ошибка при создании" + e.getMessage());  
    }  
}  
  
public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> Class, double leftX, double rightX, int pointsCount) {  
    try {  
        return Class.getConstructor(...parameterTypes: double.class, double.class, int.class).newInstance(leftX, rightX, pointsCount);  
    } catch (NoSuchMethodException e) {  
        throw new IllegalArgumentException("Не найден конструктор " + e.getMessage());  
    } catch (IllegalAccessException | IllegalArgumentException | InstantiationException | InvocationTargetException e) {  
        throw new IllegalArgumentException("Ошибка при создании" + e.getMessage());  
    }  
}  
  
public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> Class, double leftX, double rightX, double[] values) {  
    try {  
        return Class.getConstructor(...parameterTypes: double.class, double.class, double[].class).newInstance(leftX, rightX, values);  
    } catch (NoSuchMethodException e) {  
        throw new IllegalArgumentException("Не найден конструктор " + e.getMessage());  
    } catch (IllegalAccessException | IllegalArgumentException | InstantiationException | InvocationTargetException e) {  
        throw new IllegalArgumentException("Ошибка при создании" + e.getMessage());  
    }  
}
```

Методы находят в классе конструктор с соответствующими определенными параметрами (левая граница, правая граница, количество точек, массив точек Y или массив точек), корректно определяет тип и создает объект табулированной функции, после возвращает ссылку на этот объект.

В классе TabulatedFunctions перегрузил методы (tabulate, inputTabulatedFunction, readTabulatedFunction) создающие объекты табулированных функций, добавив версии, принимающие также ссылку типа Class на описание класса, объект которого требуется создать.

Сделал так, чтобы в эти методы можно было передать только ссылки на классы, реализующие интерфейс TabulatedFunction.

```
public static TabulatedFunction tabulate(Class<? extends TabulatedFunction> Class, Function function, double leftX, double rightX, int pointsCount) {  
    if(leftX >= rightX){
```

```
public static TabulatedFunction inputTabulatedFunction(Class<? extends TabulatedFunction> Class, InputStream in) throws IOException {  
    DataInputStream dis = new DataInputStream(in);
```

```
public static TabulatedFunction readTabulatedFunction(Class<? extends TabulatedFunction> Class, Reader in) throws IOException {  
    StreamTokenizer st = new StreamTokenizer(in);
```

Методы вызывают соответствующий метод createTabulatedFunction, передавая на него ссылку типа Class и массив points.

```
}  
return createTabulatedFunction(Class, points);
```

Проверил в методе main работу методов рефлексивного создания объектов, а также методов класса TabulatedFunctions, использующих создание объектов.

```
class functions.ArrayTabulatedFunction  
{(0.0 ; 0.0)(5.0 ; 0.0)(10.0 ; 0.0)}  
class functions.ArrayTabulatedFunction  
{(0.0 ; 0.0)(10.0 ; 10.0)}  
class functions.LinkedListTabulatedFunction  
{(0.0 ; 0.0)(10.0 ; 10.0)}  
class functions.LinkedListTabulatedFunction  
{(0.0 ; 0.0)(0.3141592653589793 ; 0.3090169943749474)(0.6283185307179586 ; 0.5877852522924731)(0.9424777960769379 ; 0.8090169943749475)(1.2566370614359172 ; 0.95105651  
2951535)(1.5707963267948966 ; 1.0)(1.8849555921538759 ; 0.9510565162951536)(2.199114857512855 ; 0.8090169943749475)(2.5132741228718345 ; 0.5877852522924732)(2.82743338  
230814 ; 0.3090169943749475)(3.141592653589793 ; 1.2246467991473532E-16)}  
PS C:\Users\LuckyFrut\Documents\University\Lab-7-2025> █
```